

SQL PROGRAMMING & DATABASE ANALYSIS

Complete syllabus-based study material for students and internship learners

MODULE	SQL Fundamentals, Database Design, Queries & Advanced SQL
FORMAT	Concept → Explanation → SQL Example → Practice Focus → Unit Checkpoint
PROJECT	IPL Dataset Case Study with analytical SQL questions

Read the concept, run the SQL example in your selected database, change the values, and verify the result. SQL becomes easy when every query is connected to a table and a question.

Analyst mindset: define the business question → identify tables and columns → write the query → validate the result → explain the insight.



SQL Learning Path

The syllabus progresses from installation and database concepts to DDL, constraints, CRUD queries, aggregation, joins, subqueries, sorting, grouping and a complete IPL case study.

Stage	Learning outcome
1. Setup	Install and understand MySQL, XAMPP, PostgreSQL and pgAdmin; know when each tool is used.
2. Database fundamentals	Understand databases, tables, rows, columns, schemas, keys, relationships and data types.
3. Structure	Create, modify and remove databases/tables; understand DDL and data integrity.
4. Data manipulation	Insert, select, update and delete records safely.
5. Analysis	Use aggregate/scalar functions, filtering, sorting and grouping.
6. Relational SQL	Combine tables with joins and set operations.
7. Advanced SQL	Use scalar, row, table and correlated subqueries.
8. Project	Answer business questions from an IPL dataset using reusable SQL patterns.

Every SQL exercise should answer a question. First define the business question, then identify the tables/columns, write the query, validate the result and explain the insight.



UNIT 1

Introduction and Installation Software

Roadmap to Learn SQL & Syllabus Discussion

SQL learning is easiest when it is treated as a progression: database concepts → table design → data types and keys → DDL → constraints → INSERT/SELECT/UPDATE/DELETE → functions → joins → subqueries → sorting/grouping → project analysis. The objective is not only to memorize syntax but to understand how relational data is stored and queried.

- Keep one SQL notebook for concepts and one script file for reusable queries.
- Practice the same query in more than one SQL environment when the syntax is compatible.

MySQL

MySQL is a relational database management system widely used for application and analytical workloads. You interact with it through a server and a client such as MySQL Workbench or a command-line client. The server stores databases; the client sends SQL statements and displays results.

- Understand server vs client.
- Know which database/schema is currently selected before creating tables.

Example

```
CREATE DATABASE training_db;
USE training_db;
SELECT DATABASE();
```

XAMPP

XAMPP is a local development package that can include Apache, MariaDB and related tools. In this syllabus, it is useful for learning database concepts through a local web-development environment and phpMyAdmin. Note that XAMPP commonly ships with MariaDB rather than Oracle MySQL.

- Use XAMPP for a local practice server.
- Do not confuse phpMyAdmin (a management interface) with the database engine itself.

Example

Open XAMPP Control Panel → start MySQL/MariaDB → open phpMyAdmin → create a database → create a table.

PostgreSQL & pgAdmin

PostgreSQL is an open-source relational database system with strong SQL support and advanced data types. pgAdmin is a graphical administration tool used to connect to PostgreSQL servers, create databases and run SQL queries.

- Learn the difference between the PostgreSQL server and pgAdmin client.
- Keep credentials and connection settings private.

Example

```
CREATE DATABASE training_db;
-- Then connect to training_db in pgAdmin and run SQL statements.
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 1. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 2

Fundamentals of SQL

Introduction to SQL

SQL (Structured Query Language) is used to define, manipulate and query data in relational database systems. SQL statements are declarative: you describe what result you want, while the database engine decides how to execute the request.

- SELECT retrieves data.
- DDL defines structures.
- DML changes data.
- Constraints protect data integrity.

Example

```
SELECT product_name, price
FROM products
WHERE price > 1000;
```

Database Fundamentals

A database is an organized collection of related data. A relational database stores data in tables. A table contains rows (records) and columns (attributes). A primary key uniquely identifies rows, while foreign keys connect related tables.

- Understand database → schema → table → row/column.
- Think about relationships before writing complex joins.

Cheat Code / Quick Reference

A practical SQL cheat sheet should cover the statement patterns used repeatedly: CREATE DATABASE, CREATE TABLE, INSERT, SELECT, WHERE, ORDER BY, GROUP BY, HAVING, JOIN, UPDATE and DELETE. The goal is to recognize patterns quickly, not blindly copy them.

Example

```
SELECT column1, column2
FROM table_name
WHERE condition
GROUP BY column1
HAVING aggregate_condition
ORDER BY column1 DESC;
```

Practice Dataset

A good practice dataset contains multiple related tables, enough rows to make filtering meaningful, and realistic business questions. For this course, use a small ecommerce schema or the IPL dataset introduced later.

- Start with 10-20 rows while learning syntax.
- Move to larger datasets after you understand the query.

PDF Notes, Books & Learning Websites

Use documentation and structured references to clarify syntax, but verify examples against the database engine you are using. SQL has a common core, but functions, auto-increment syntax, data types and some DDL features can differ between MySQL, PostgreSQL and other systems.

- Keep notes on engine-specific syntax.
- When a query fails, read the exact error message before changing random parts of the query.

Interview Questions

Interview preparation should test understanding: What is a primary key? Difference between WHERE and HAVING? INNER vs LEFT JOIN? What is a subquery? How do NULLs behave? How do you find the second-highest value? The student should be able to explain both the syntax and the reasoning.

- Practice explaining queries in plain language.

- Do not memorize answers without running examples.

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 2. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 3

Case Study with Example: Database and Tables

Create Database

CREATE DATABASE creates a new database. After creating it, select/connect to it before creating tables. In PostgreSQL, you normally connect to the database separately; in MySQL, USE selects the database for the session.

Example

```
CREATE DATABASE ecommerce_db;
USE ecommerce_db;
```

Drop Database

DROP DATABASE permanently removes the database and its objects. It is destructive and should be used only when you are certain the data is disposable.

- Never run destructive commands on production data without authorization and backups.
- Use a training database for practice.

Example

```
DROP DATABASE ecommerce_db;
```

Create Table

CREATE TABLE defines columns, data types and optional constraints. A good table design identifies each attribute, chooses a suitable type and defines keys and integrity rules.

Example

```
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE,
  city VARCHAR(80)
);
```

Insert Data

INSERT adds rows to a table. Explicitly naming columns makes a statement clearer and safer because the value order is visible.

Example

```
INSERT INTO customers (customer_id, customer_name, email, city)
VALUES (1, 'Asha', 'asha@example.com', 'Pune');
```

Drop Table

DROP TABLE removes the table and its stored data. It is different from DELETE, which removes rows while leaving the table structure.

- DROP TABLE removes structure and data.
- DELETE removes selected rows; TRUNCATE removes all rows while retaining the table structure.

Example

```
DROP TABLE customers;
```

Case Study with XAMPP

Create an ecommerce database in XAMPP/phpMyAdmin. Start with customers and products, then create orders and order_items. Practice creating tables, inserting sample records and running SELECT queries.

Example

```
CREATE TABLE products (
  product_id INT PRIMARY KEY,

  product_name VARCHAR(100) NOT NULL,
  price DECIMAL(10,2) NOT NULL
);
```

Case Study with MySQL

Repeat the same ecommerce schema using MySQL. The learning objective is to understand the server/client workflow and to build tables in a controlled database.

- Create database.
- Create tables in dependency order.
- Insert parent rows before child rows when foreign keys are enabled.

Case Study with PostgreSQL

Repeat the schema in PostgreSQL using pgAdmin. PostgreSQL's syntax overlaps heavily with standard SQL but has different auto-generated key syntax and a richer type system.

Example

```
CREATE TABLE products (
  product_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
  price NUMERIC(10,2) NOT NULL
);
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 3. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 4

Data Types and Keys

SQL Data Types

Data types tell the database what kind of values a column can store. Choosing the right type improves validation, storage and query behavior. Exact names vary by database engine.

- Numeric: INTEGER/INT, BIGINT, DECIMAL/NUMERIC, FLOAT/REAL.
- Character: CHAR, VARCHAR, TEXT.
- Date/time: DATE, TIME, TIMESTAMP.
- Boolean: BOOLEAN/BOOL where supported.
- Binary/large objects: BLOB and engine-specific types.
- PostgreSQL also provides advanced types such as JSONB, ARRAY, UUID and range types.

Numeric Types

Integer types are appropriate for counts and identifiers; DECIMAL/NUMERIC is preferred for exact monetary values; floating-point types are useful for scientific measurements where small representation differences are acceptable.

- Avoid FLOAT for currency when exact decimal arithmetic is required.
- Choose size/range based on expected values.

Example

```
price DECIMAL(10,2)
quantity INT
ratio DOUBLE
```

Character Types

CHAR stores fixed-length character data, while VARCHAR stores variable-length text up to a declared limit. TEXT is used for longer text in systems that support it.

Example

```
customer_name VARCHAR(100)
country_code CHAR(2)
notes TEXT
```

Date and Time Types

DATE stores calendar dates. TIME stores a time of day. TIMESTAMP stores date and time. Some systems also distinguish time zone-aware timestamps.

- Use date/time types rather than storing dates as arbitrary strings.
- Timezone requirements should be considered for real systems.

Example

```
order_date DATE
created_at TIMESTAMP
```

ER / Database Diagram

An Entity Relationship (ER) diagram represents entities/tables, their attributes and relationships. It helps you design the schema before writing CREATE TABLE statements.

- Entity = table concept.
- Attribute = column.
- Relationship = connection between entities.
- Keys identify and connect records.

Amazon Ecommerce Database Example

A simplified ecommerce model can contain Customer, Product, Order, OrderItem, Payment and Category entities. An Order belongs to one Customer; an Order can contain many OrderItems; each OrderItem refers to one Product.

- Customer 1—N Order.
- Order 1—N OrderItem.
- Product 1—N OrderItem.
- Category 1—N Product in a simple model.

Keys

A key is an attribute or set of attributes used to identify or relate records. Primary keys uniquely identify rows. Foreign keys reference keys in another table. Candidate keys are possible unique identifiers; a composite key uses multiple columns.

Example

```
CREATE TABLE order_items (
order_id INT,
product_id INT,
quantity INT,
PRIMARY KEY (order_id, product_id)
);
```

Cardinality of Relationships

Cardinality describes how many records in one entity can relate to records in another: one-to-one, one-to-many and many-to-many. Many-to-many relationships are normally implemented using a junction/bridge table.

- 1:1 — one person to one profile.
- 1:N — one customer to many orders.
- M:N — many orders contain many products, implemented through order items.

UNIT CHECKPOINT — Complete 5-10 queries for UNIT 4. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 5

DDL Commands for SQL

DDL Overview

Data Definition Language defines database objects. The core syllabus commands are CREATE, DROP and TRUNCATE. DDL behavior and transaction support can vary by database engine.

- CREATE builds an object.
- DROP removes an object.
- TRUNCATE removes all rows from a table while retaining the table definition.

CREATE Database and Table

CREATE DATABASE creates a database. CREATE TABLE defines a table and its columns, types and constraints.

Example

```
CREATE DATABASE sales_db;
```

```
CREATE TABLE sales (
  sale_id INT PRIMARY KEY,
  amount DECIMAL(10,2)
);
```

TRUNCATE

TRUNCATE removes all rows from a table quickly while retaining the table structure. It is different from DELETE because it is intended to empty the table rather than selectively remove rows.

- Use DELETE when a WHERE condition is needed.
- Check engine-specific transaction and identity behavior.

Example

```
TRUNCATE TABLE sales;
```

DROP

DROP removes a database object. Dropping a table removes its definition and data; dropping a database removes the database and contained objects.

Example

```
DROP TABLE sales;
```

Data Integrity

Data integrity means data remains accurate, valid and consistent. Keys, constraints, correct data types and careful transactions help protect integrity.

- Entity integrity: primary key uniqueness/non-null requirement.
- Referential integrity: foreign keys must reference valid parent rows unless the relationship allows NULL.
- Domain integrity: values must fit the intended type/rules.

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 5. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 6

Constraints

NOT NULL

NOT NULL prevents a column from storing SQL NULL. It is appropriate when a value is mandatory for every row.

Example


```
customer_name VARCHAR(100) NOT NULL
```

UNIQUE

UNIQUE prevents duplicate values in a constrained column or column combination. NULL behavior can vary by database system, so verify engine-specific semantics.

Example

```
email VARCHAR(150) UNIQUE
```

PRIMARY KEY

A primary key uniquely identifies each row. A table has one primary key constraint, which may consist of one or multiple columns.

- Primary keys should be stable and unique.
- Composite primary keys contain multiple columns.

Example

```
customer_id INT PRIMARY KEY
```

AUTO INCREMENT / Identity

MySQL commonly uses AUTO_INCREMENT for generated numeric keys. PostgreSQL commonly uses IDENTITY columns. These mechanisms generate values rather than requiring the user to provide each identifier.

- Syntax differs between MySQL and PostgreSQL.
- Generated identifiers are not business meaning; do not encode business rules into them.

Example

```
CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100)
);
```

FOREIGN KEY

A foreign key enforces a relationship by requiring a value to match a referenced key in the parent table, unless NULL is allowed.

Example

```
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

CHECK

CHECK defines a Boolean condition that inserted or updated values must satisfy in database systems that enforce the constraint.

Example

```
age INT CHECK (age >= 18)
```

DEFAULT

DEFAULT supplies a value when an INSERT does not specify the column. It is useful for timestamps, statuses and other common starting values.

Example

```
status VARCHAR(20) DEFAULT 'Pending'
```

Referential Actions

When a referenced parent row changes or is deleted, referential actions define what happens to child rows. Common actions include RESTRICT, CASCADE, SET NULL and SET DEFAULT, subject to database support and column requirements.

- RESTRICT: prevent the parent action when dependent rows exist.
- CASCADE: propagate the change.
- SET NULL: clear the foreign key, requiring it to allow NULL.
- SET DEFAULT: use the column default where supported.

Example

```
FOREIGN KEY (customer_id)
REFERENCES customers(customer_id)
ON DELETE CASCADE
ON UPDATE CASCADE
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 6. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 7

ALTER TABLE

ADD Columns

ALTER TABLE ... ADD COLUMN changes an existing table by adding a new column. Consider whether existing rows can satisfy a NOT NULL requirement before adding it.

Example

```
ALTER TABLE customers
ADD COLUMN phone VARCHAR(20);
```

DELETE Columns

A column can be removed with DROP COLUMN in systems that support it. Dropping a column is destructive to the data stored in that column.

- Check dependencies before dropping columns.
- Use backups/version control for schema changes in real systems.

Example

```
ALTER TABLE customers
DROP COLUMN phone;
```

MODIFY Columns

Column definitions can be changed to adjust data type, size or other properties. Syntax differs between engines: MySQL uses MODIFY/CHANGE patterns, while PostgreSQL commonly uses ALTER COLUMN ... TYPE.

- Always check existing values before narrowing a type or length.
- Document schema changes.

Example

```
ALTER TABLE customers
MODIFY COLUMN customer_name VARCHAR(150);
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 7. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 8

Adding, Editing and Deleting Constraints

ADD Constraints

Constraints can be added after table creation using ALTER TABLE. This is useful when an existing schema needs strong validation.

Example

```
ALTER TABLE customers
ADD CONSTRAINT uq_customer_email UNIQUE (email);
```

DELETE Constraints

Constraints can be removed when requirements change. The exact syntax depends on the database engine and constraint type.

- In MySQL, some constraint types have specialized DROP syntax such as DROP FOREIGN KEY or DROP INDEX for unique indexes.

Example

```
ALTER TABLE customers
DROP CONSTRAINT uq_customer_email;
```

EDIT Constraints

Constraints are normally changed by dropping the old definition and creating a new definition, because not every database supports direct modification of every constraint property.

- Check current constraints first.
- Change one rule at a time and test existing data.
- Be aware that existing rows may violate a newly added constraint.

Problems in MySQL

Common constraint problems include duplicate values, missing parent keys, invalid CHECK conditions, attempts to insert NULL into NOT NULL columns, and foreign-key errors caused by mismatched definitions or missing referenced rows.

Example

```
INSERT INTO customers(customer_id, customer_name)
VALUES (1, 'Asha');
-- Running the same primary key value again causes a duplicate-key error.
```

Drop Constraint

Dropping a constraint changes the integrity guarantees of a table. Do this only when the rule is no longer required and the impact is understood.

- Do not drop constraints merely to make an error disappear.
- Fix the underlying data or schema design when possible.

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 8. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 9

INSERT Query

INSERT Query

INSERT adds one or more rows. Always specify target columns when possible.

Example

```
INSERT INTO products (product_id, product_name, price)
VALUES (1, 'Laptop', 55000.00);
```

INSERT Query Variations

You can insert using all relevant columns, a selected subset with defaults, or values derived from a SELECT query. Syntax varies slightly by database.

- Column order in the VALUES list must match the named columns.
- Defaults apply when the column is omitted.

Example

```
INSERT INTO products (product_name, price)
VALUES ('Mouse', 800.00);
```

INSERT Multiple Values

Multiple rows can be inserted in a single statement, which is concise and often more efficient than separate INSERT statements.

Example

```
INSERT INTO products (product_id, product_name, price)
VALUES
(2, 'Keyboard', 1200.00),
(3, 'Monitor', 15000.00),
(4, 'Webcam', 2500.00);
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 9. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 10

SELECT Query

SELECT All Columns

SELECT retrieves data. SELECT * returns all columns, which is convenient for exploration but less explicit for production queries.

Example

```
SELECT * FROM products;
```

Filter Columns

Select only the columns needed for the question. This improves readability and avoids unnecessary data transfer.

Example

```
SELECT product_name, price
FROM products;
```

Alias / Renaming Columns

Aliases rename columns or tables for the duration of a query. They make output easier to read and joins easier to write.

Example

```
SELECT product_name AS Product, price AS Price
FROM products AS p;
```

Expressions Using Columns

SQL can calculate new values directly in SELECT. Expressions can combine columns, arithmetic and functions.

Example

```
SELECT product_name,
price,
price * 1.18 AS price_with_tax
FROM products;
```

Constant Values

A query can include literal values as columns. This is useful for labels, fixed categories and testing.

Example

```
SELECT product_name, 'India' AS country
FROM products;
```

DISTINCT

DISTINCT removes duplicate combinations from the selected output. It applies to the entire selected column combination.

Example

```
SELECT DISTINCT city
FROM customers;
```

DISTINCT Combinations

When several columns are selected with DISTINCT, uniqueness is evaluated across the combination, not each column independently.

Example

```
SELECT DISTINCT city, state
FROM customers;
```

WHERE Clause

WHERE filters rows before grouping and aggregation. It supports comparison, logical operators and many predicates.

Example

```
SELECT *
FROM products
WHERE price > 1000 AND price < 50000;
```

BETWEEN

BETWEEN checks whether a value falls within an inclusive range in typical SQL implementations. It can improve readability compared with two comparisons.

Example

```
SELECT *
FROM products
WHERE price BETWEEN 1000 AND 50000;
```

IN and NOT IN

IN tests membership in a list or subquery result. NOT IN excludes matching values, but NULL behavior requires special care.

- Prefer IN over long chains of OR when testing the same column.
- Understand NULL behavior before using NOT IN with subqueries.

Example

```
SELECT *
FROM customers
WHERE city IN ('Pune', 'Delhi', 'Mumbai');
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 10. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 11

UPDATE Query

Update Rows

UPDATE changes existing rows. A WHERE clause is essential when only selected rows should change.

- Run a SELECT with the same WHERE condition before UPDATE to verify the target rows.
- Use transactions when appropriate.

Example

```
UPDATE products
SET price = 1250
WHERE product_id = 2;
```

Update Multiple Columns

Several columns can be changed in one UPDATE statement by separating assignments with commas.

Example

```
UPDATE customers
SET city = 'Pune',
    email = 'new@example.com'
WHERE customer_id = 1;
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 11. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 12

DELETE Query

DELETE Rows

DELETE removes rows from a table. With WHERE, only matching records are deleted. Without WHERE, all rows are deleted.

- Preview the target rows using SELECT first.
- A missing WHERE clause can remove every row.

Example

```
DELETE FROM products
WHERE product_id = 4;
```

Deletion Based on Multiple Conditions

Multiple conditions can be combined with AND/OR. Parentheses are important when mixing AND and OR because SQL evaluates operators according to precedence rules.

Example

```
DELETE FROM customers
WHERE city = 'Pune'
AND customer_id > 100;
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 12. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 13

Aggregate Functions

MAX() & MIN()

MAX returns the largest value and MIN returns the smallest. They are aggregate functions because they summarize multiple rows into a result.

Example

```
SELECT MAX(price) AS highest_price,
       MIN(price) AS lowest_price
FROM products;
```

AVG()

AVG calculates the arithmetic mean of non-NULL numeric values. The result can be affected by outliers and missing values.

Example

```
SELECT AVG(price) AS average_price
FROM products;
```

SUM()

SUM adds numeric values across rows. It is frequently used for revenue, quantities and other totals.

Example

```
SELECT SUM(quantity * price) AS revenue
FROM order_items;
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 13. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 14

Scalar Functions

ABS()

ABS returns the absolute value, removing the sign. It is useful when magnitude matters more than direction.

Example

```
SELECT ABS(-125) AS magnitude;
```

ROUND()

ROUND rounds a numeric expression to a chosen number of decimal places. It is useful for presentation and controlled calculations.

Example

```
SELECT ROUND(1234.5678, 2) AS rounded_value;
```

CEIL() & FLOOR()

CEIL (or CEILING) rounds a number upward to the next integer; FLOOR rounds downward to the previous integer.

Example

```
SELECT CEIL(12.1) AS up_value,
       FLOOR(12.9) AS down_value;
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 14. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 15

SQL Joins

What Are SQL Joins?

A JOIN combines rows from related tables using a condition, usually matching a primary key to a foreign key. Joins allow normalized data to be analyzed as a combined result.

Example

```
SELECT o.order_id, c.customer_name
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id;
```

Cross Join — Cartesian Product

CROSS JOIN pairs every row from the first table with every row from the second. If one table has 3 rows and another has 4, the result has 12 combinations.

- Use deliberately; accidental Cartesian products can create huge results.

Example

```
SELECT c.customer_name, p.product_name
FROM customers c
CROSS JOIN products p;
```

INNER JOIN

INNER JOIN returns only rows that have matching values in both joined tables.

Example

```
SELECT o.order_id, c.customer_name
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;
```

LEFT JOIN

LEFT JOIN keeps every row from the left table and adds matching right-table values. If no match exists, right-side columns become NULL.

- Useful for finding customers with no orders by filtering WHERE o.order_id IS NULL.

Example

```
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

RIGHT JOIN

RIGHT JOIN keeps every row from the right table and matching rows from the left. It is logically equivalent to reversing the table order in a LEFT JOIN.

Example

```
SELECT c.customer_name, o.order_id
FROM customers c
RIGHT JOIN orders o
ON c.customer_id = o.customer_id;
```

FULL OUTER JOIN

FULL OUTER JOIN returns matched rows plus unmatched rows from both sides. PostgreSQL supports FULL OUTER JOIN; MySQL does not provide it as a native join type, so a UNION-based approach is often used.

- Check engine support before using FULL OUTER JOIN.

Example

```
SELECT *
FROM customers c
FULL OUTER JOIN orders o
ON c.customer_id = o.customer_id;
```

Set Operations

UNION combines result sets and removes duplicates; UNION ALL keeps duplicates. INTERSECT returns common rows and EXCEPT returns rows in the first result not in the second, subject to database support.

- The queries being combined must have compatible column counts and types.
- Use UNION ALL when duplicates are meaningful or when deduplication is unnecessary.

Example

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers;
```

Self-Join

A self-join joins a table to itself. It is useful for hierarchical relationships such as employees and managers or comparing rows within the same entity.

Example

```
SELECT e.employee_name, m.employee_name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.employee_id;
```

Filtering Columns and Rows in Joins

Filter columns in SELECT and filter rows with WHERE or JOIN conditions. Be careful: placing a condition on the right table in WHERE after a LEFT JOIN can effectively turn it into an INNER JOIN for those rows.

- Put relationship logic in ON.
- Put final row filtering in WHERE when that matches the intended semantics.

Example

```
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE c.city = 'Pune';
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 15. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 16

Subquery

What Is a Subquery?

A subquery is a query nested inside another query. It can supply a single value, a set of values, rows or a derived table.

Example

```
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

Scalar Subquery

A scalar subquery returns exactly one value. It can be used where a single expression is expected.

- If the subquery returns multiple rows where one value is expected, the query fails.

Example

```
SELECT product_name, price
FROM products
WHERE price = (SELECT MAX(price) FROM products);
```

Row Subquery

A row subquery returns one row with multiple columns and can be compared to a row constructor where supported.

- Exact row-subquery syntax and LIMIT behavior vary by database.
- Understand the shape of the subquery result.

Example

```
SELECT *
FROM products
WHERE (category_id, price) =
(SELECT category_id, MAX(price)
FROM products
GROUP BY category_id
ORDER BY MAX(price) DESC
LIMIT 1);
```

Table Subquery

A table subquery produces a derived table that the outer query can treat like a temporary result set. It is useful for multi-step analysis.

Example

```
SELECT x.category_id, x.avg_price
FROM (
  SELECT category_id, AVG(price) AS avg_price
  FROM products
  GROUP BY category_id
) x
WHERE x.avg_price > 1000;
```

Correlated Subquery

A correlated subquery refers to a column from the outer query. The inner query is logically evaluated for each outer row, so it can be powerful but may be more expensive than a join or window function.

- Use correlated subqueries when they make the logic clearer.
- For large data, compare performance with joins or window functions.

Example

```
SELECT p.product_name, p.price
FROM products p
WHERE p.price > (
```

UNIT 17

Sorting

Fundamentals of SQL Sorting

ORDER BY sorts the final result by one or more columns. ASC is ascending and DESC is descending. Sorting is applied after the result rows are formed.

Example

```
SELECT product_name, price
FROM products
ORDER BY price ASC;
```

Sorting Top 10 Data

To retrieve the highest 10 records, sort descending and then limit the result. LIMIT is common in MySQL/PostgreSQL; SQL Server uses TOP or OFFSET/FETCH.

- Always define the ranking column explicitly.
- Add a tie-breaker when deterministic order matters.

Example

```
SELECT product_name, price
FROM products
ORDER BY price DESC
LIMIT 10;
```

Sorting Descending Orders

DESC sorts from highest to lowest for numeric values and reverse alphabetical order for text. You can sort by multiple columns.

Example

```
SELECT customer_name, city
FROM customers
ORDER BY city ASC, customer_name DESC;
```

Sorting Functions in Datasets

Sorting becomes useful during exploratory analysis: top products, lowest sales, newest orders, highest strike rate, or customers with the most orders. Combine ORDER BY with filters, aggregates and limits.

Example

```
SELECT product_id, SUM(quantity) AS total_qty
FROM order_items
GROUP BY product_id
ORDER BY total_qty DESC
LIMIT 10;
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 17. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 18

Grouping

Fundamentals of SQL Grouping

GROUP BY combines rows into groups so aggregate functions can calculate summaries per category. Any selected non-aggregated column normally needs to be included in GROUP BY.

Example

```
SELECT city, COUNT(*) AS customers
FROM customers
GROUP BY city;
```

Grouping Average Function

AVG can be combined with GROUP BY to calculate an average for each group rather than one average for the whole table.

Example

```
SELECT category_id, AVG(price) AS avg_price
FROM products
GROUP BY category_id;
```

GROUP BY Multiple Columns

Grouping by multiple columns creates groups for each unique combination. This is useful for summaries such as sales by region and month.

Example

```
SELECT region, product_category,
SUM(sales) AS total_sales
FROM sales
GROUP BY region, product_category;
```

Sorting Grouped Results

Grouped results can be sorted by an aggregate alias or expression. This is how you find top-performing categories or regions.

Example

```
SELECT region, SUM(sales) AS total_sales
FROM sales
GROUP BY region
ORDER BY total_sales DESC;
```

HAVING vs WHERE

WHERE filters individual rows before grouping. HAVING filters groups after aggregation. This distinction is essential in analytical SQL.

- Use WHERE for row-level conditions.
- Use HAVING for aggregate/group-level conditions.

Example

```
SELECT region, SUM(sales) AS total_sales
FROM sales
WHERE sales_date >= '2026-01-01'
GROUP BY region
HAVING SUM(sales) > 100000
ORDER BY total_sales DESC;
```

UNIT CHECKPOINT — Complete 5–10 queries for UNIT 18. For every query, write one sentence explaining what the database is being asked to return or change.

UNIT 19

Project & Case Study — Indian Premier League (IPL Dataset)

Project Objective

The IPL case study converts the syllabus into real analytical questions. The student should first inspect the dataset, identify the batting columns, validate data types and then build each answer using aggregation, filtering, sorting and subqueries. The exact column names may differ by IPL dataset, so map the supplied dataset columns before executing the final queries.

Question SQL skills practiced

Top 5 batsmen in IPL GROUP BY, SUM, ORDER BY DESC, LIMIT/TOP

2nd highest 6-hitters GROUP BY, SUM, DISTINCT/ranking logic, sorting

M.S. Dhoni & Virat Kohli vs all teams

WHERE, GROUP BY, joins if team/opponent data is split

Top 10 batsmen with centuries Subqueries/aggregation, HAVING, sorting

Top 5 batsmen by strike rate SUM, conditional/derived calculations, filtering minimum balls

5 batsmen who played at least 1000 balls

GROUP BY, SUM, HAVING, ORDER BY

Step 1 — Understand the Dataset

Before writing the project queries, inspect the schema. A common ball-by-ball IPL dataset may contain fields such as batsman, bowler, batting team, bowling team, runs, extras and match identifiers. A separate batting-summary dataset may already contain runs, balls, fours, sixes and strike rate. Do not assume these names; map the actual file columns first.

```
-- Example inspection
SELECT *
FROM ipl_data
LIMIT 10;
```

```
-- Inspect unique batsmen
SELECT DISTINCT batsman
FROM ipl_data
ORDER BY batsman;
```

Step 2 — Top 5 Batsmen by Runs

If the dataset is ball-by-ball, total runs should be calculated by summing batsman runs and grouping by batsman. If your dataset contains a ready-made runs column in a batting table, use that column instead. Always confirm whether extras are included in the runs field.

```
SELECT batsman,
SUM(batsman_runs) AS total_runs
FROM ipl_deliveries
GROUP BY batsman
ORDER BY total_runs DESC
LIMIT 5;
```

Step 3 — 2nd Highest Six-Hitter

Count sixes by batsman, sort from highest to lowest and select the second position. If ties matter, use a ranking approach rather than assuming the second physical row is the second-highest distinct value.

```
WITH six_counts AS (
  SELECT batsman,
  SUM(CASE WHEN batsman_runs = 6 THEN 1 ELSE 0 END) AS sixes
  FROM ipl_deliveries
  GROUP BY batsman
)
SELECT batsman, sixes
FROM six_counts

ORDER BY sixes DESC
LIMIT 1 OFFSET 1;
```

Step 4 — Dhoni & Kohli Performance Against Every IPL Team

Filter the batsman names, group by batsman and opponent/bowling team, and calculate runs, balls and optionally strike rate. The opponent column name depends on the dataset design.

```
SELECT batsman,
bowling_team AS opponent,
SUM(batsman_runs) AS runs,
COUNT(*) AS balls_faced,
ROUND(100.0 * SUM(batsman_runs) / NULLIF(COUNT(*),0), 2) AS strike_rate
FROM ipl_deliveries
WHERE batsman IN ('MS Dhoni', 'V Kohli')
GROUP BY batsman, bowling_team
ORDER BY batsman, runs DESC;
```

Step 5 — Top 10 Batsmen with Centuries Using a Subquery

A century means at least 100 runs in a single innings. For a ball-by-ball dataset, first aggregate by batsman and match/innings, identify innings with 100 or more, then count those innings per batsman. A subquery or CTE makes this multi-step logic readable.

```
WITH innings_scores AS (
  SELECT match_id,
  batsman,
  SUM(batsman_runs) AS innings_runs
  FROM ipl_deliveries
  GROUP BY match_id, batsman
),
centuries AS (
  SELECT batsman, COUNT(*) AS centuries
  FROM innings_scores
  WHERE innings_runs >= 100
  GROUP BY batsman
)
SELECT batsman, centuries
FROM centuries
ORDER BY centuries DESC
LIMIT 10;
```

Step 6 — Top 5 Batsmen by Strike Rate

Strike rate is typically calculated as runs divided by balls faced multiplied by 100. A minimum-ball threshold is important because a player who faced very few balls could otherwise rank unusually high. Apply a threshold that matches the project requirement.

```
SELECT batsman,
SUM(batsman_runs) AS runs,
COUNT(*) AS balls,
ROUND(100.0 * SUM(batsman_runs) / NULLIF(COUNT(*),0), 2) AS strike_rate
FROM ipl_deliveries
GROUP BY batsman
HAVING COUNT(*) >= 100
ORDER BY strike_rate DESC
LIMIT 5;
```

Step 7 — Batsmen Who Played at Least 1000 Balls

Group by batsman, calculate total balls faced, keep only players with at least 1000 balls using HAVING, then sort the result. This is a direct application of grouping and aggregate filtering.

```
SELECT batsman,
COUNT(*) AS balls_faced,
SUM(batsman_runs) AS runs
FROM ipl_deliveries
GROUP BY batsman
HAVING COUNT(*) >= 1000
ORDER BY balls_faced DESC
LIMIT 5;
```

SQL Quick Reference & Interview Preparation

Core Query Pattern

```
SELECT [DISTINCT] columns_or_expressions
FROM table
[JOIN other_table ON join_condition]
[WHERE row_condition]
[GROUP BY grouping_columns]
[HAVING group_condition]
[ORDER BY sort_columns]
[LIMIT number];
```

Most Important Differences

ConceptRemember

WHERE vs HAVINGWHERE filters rows before grouping; HAVING filters groups after aggregation.

DELETE vs TRUNCATE vsDROP

DELETE removes rows; TRUNCATE empties a table while retaining its structure; DROP removes the objectitself.

PRIMARY KEY vs UNIQUEPrimary key identifies the row and is the table's main key; UNIQUE prevents duplicate constrained values.

INNER vs LEFT JOININNER keeps matches only; LEFT keeps every left-side row and matching right-side rows.

UNION vs UNION ALLUNION removes duplicate result rows; UNION ALL preserves them.

Subquery vs JOINBoth can solve relational problems; choose the clearest and, for large workloads, compare performance.

COUNT(*) vs COUNT(column)COUNT(*) counts rows; COUNT(column) counts non-NULL values in that column.

NULLNULL means missing/unknown, not zero or an empty string. Use IS NULL / IS NOT NULL rather than = NULL.

Interview Practice Questions

- What is SQL and why is it used in data analysis?
- What is the difference between a database, table, row and column?
- What is a primary key? Can a primary key contain NULL?
- What is a foreign key and why is referential integrity important?
- Explain INNER JOIN and LEFT JOIN with an example.

- What is the difference between WHERE and HAVING?
- How would you find the top 10 products by sales?
- How would you find the second-highest salary?
- What is a correlated subquery?
- When would you use GROUP BY?
- What is the difference between DELETE, TRUNCATE and DROP?
- What happens if you UPDATE or DELETE without a WHERE clause?
- How do NULL values affect aggregate functions?
- What is a many-to-many relationship and how is it implemented?
- How would you validate that a SQL query returns the expected result?



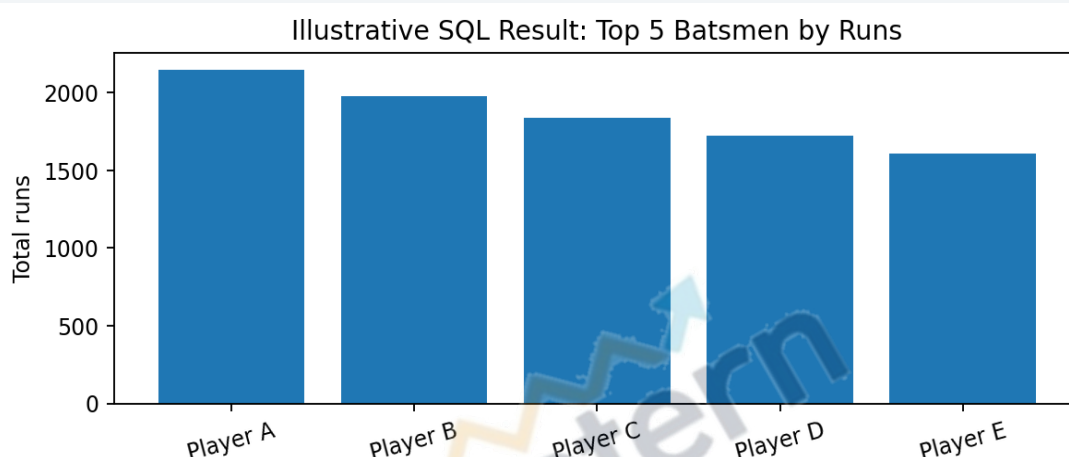
SQL ANALYTICS LAB

From SQL Query → Result → Chart

These examples use clearly labelled illustrative practice values. The source IPL project instructs learners to inspect the real dataset and map the actual column names before executing final queries.

1. Top 5 Batsmen by Runs

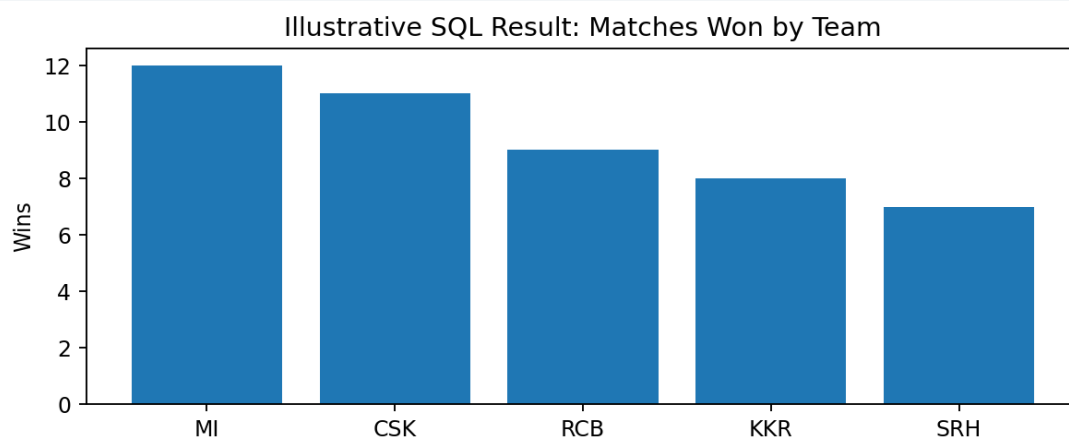
```
SELECT batsman,
SUM(batsman_runs) AS total_runs
FROM ipl_deliveries
GROUP BY batsman
ORDER BY total_runs DESC
LIMIT 5;
```



What to observe: GROUP BY creates one result per batsman; SUM calculates total runs; ORDER BY ranks the results; LIMIT keeps the top five.

2. Team Performance

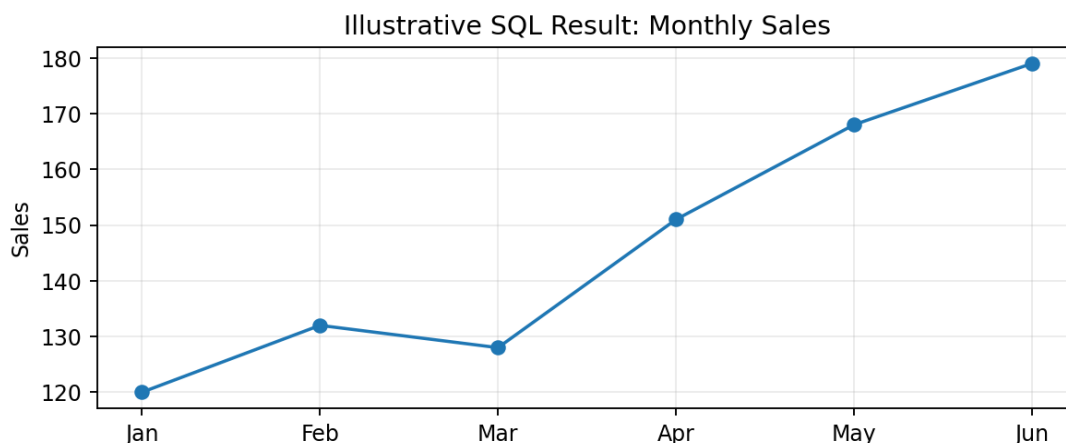
```
SELECT team,
COUNT(*) AS matches_won
FROM team_results
WHERE result = 'Won'
GROUP BY team
ORDER BY matches_won DESC;
```



SQL ANALYTICS LAB — MORE VISUALS

3. Monthly Sales Trend

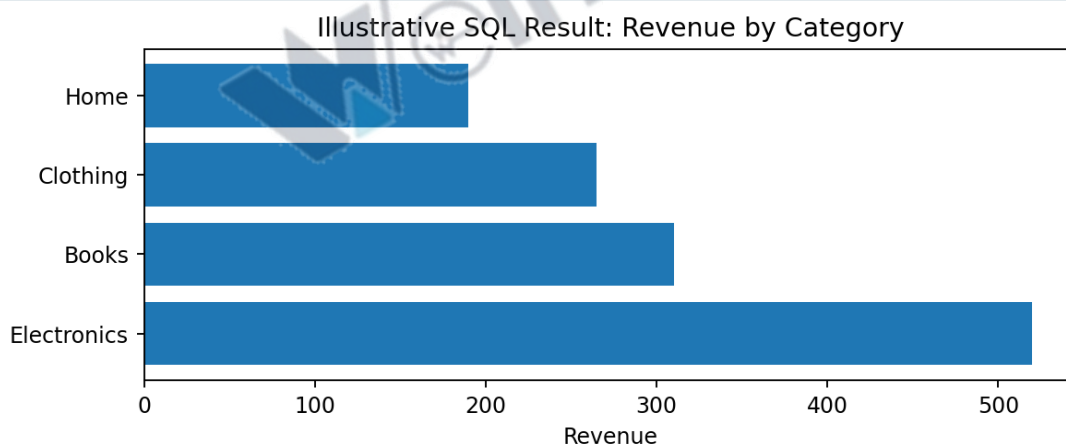
```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
SUM(amount) AS total_sales
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```



Use a line chart when the SQL result represents an ordered time trend.

4. Revenue by Category

```
SELECT category,
SUM(quantity * price) AS revenue
FROM order_items
GROUP BY category
ORDER BY revenue DESC;
```



Use a bar chart when comparing independent categories. The chart values here are illustrative practice data.

SQL QUERY → DATA ANALYSIS WORKFLOW

A practical workflow for students

1	Question	What business question are you trying to answer?
2	Schema	Which table and columns contain the required information?
3	Query	Use SELECT, WHERE, GROUP BY, JOIN, HAVING and ORDER BY as required.
4	Validate	Check row counts, sample records, NULLs and unexpected duplicates.
5	Visualize	Export or aggregate the SQL result and choose an appropriate chart.
6	Explain	Write two or three sentences explaining the result and business meaning.

Practice challenge

Take the IPL case study questions from Unit 19 and produce one SQL result table plus one chart for at least three questions. For each chart, write why that chart type is appropriate.

SQL answers the data question; a chart makes the pattern easier to see. Always verify the numbers in the result set before presenting the visualization.



SQL QUICK REFERENCE & INTERVIEW PREPARATION

Core Query Pattern

```
SELECT [DISTINCT] columns_or_expressions
FROM table
[JOIN other_table ON join_condition]
[WHERE row_condition]
[GROUP BY grouping_columns]
[HAVING group_condition]
[ORDER BY sort_columns]
[LIMIT number];
```

Most Important Differences

WHERE vs HAVING	WHERE filters rows before grouping; HAVING filters groups after aggregation.
DELETE vs TRUNCATE vs DROP	DELETE removes rows; TRUNCATE empties a table while retaining its structure; DROP removes the object itself.
PRIMARY KEY vs UNIQUE	Primary key identifies the row and is the table's main key; UNIQUE prevents duplicate constrained values.
INNER vs LEFT JOIN	INNER keeps matches only; LEFT keeps every left-side row and matching right-side rows.
UNION vs UNION ALL	UNION removes duplicate result rows; UNION ALL preserves them.
Subquery vs JOIN	Both can solve relational problems; choose the clearest and, for large workloads, compare performance.
COUNT(*) vs COUNT(column)	COUNT(*) counts rows; COUNT(column) counts non-NULL values in that column.
NULL	NULL means missing/unknown, not zero or an empty string. Use IS NULL / IS NOT NULL rather than = NULL.

Interview Practice Questions

01. What is SQL and why is it used in data analysis?
02. What is the difference between a database, table, row and column?
03. What is a primary key? Can a primary key contain NULL?
04. What is a foreign key and why is referential integrity important?
05. Explain INNER JOIN and LEFT JOIN with an example.
06. What is the difference between WHERE and HAVING?
07. How would you find the top 10 products by sales?
08. How would you find the second-highest salary?
09. What is a correlated subquery?
10. When would you use GROUP BY?
11. What is the difference between DELETE, TRUNCATE and DROP?
12. What happens if you UPDATE or DELETE without a WHERE clause?
13. How do NULL values affect aggregate functions?
14. What is a many-to-many relationship and how is it implemented?
15. How would you validate that a SQL query returns the expected result?